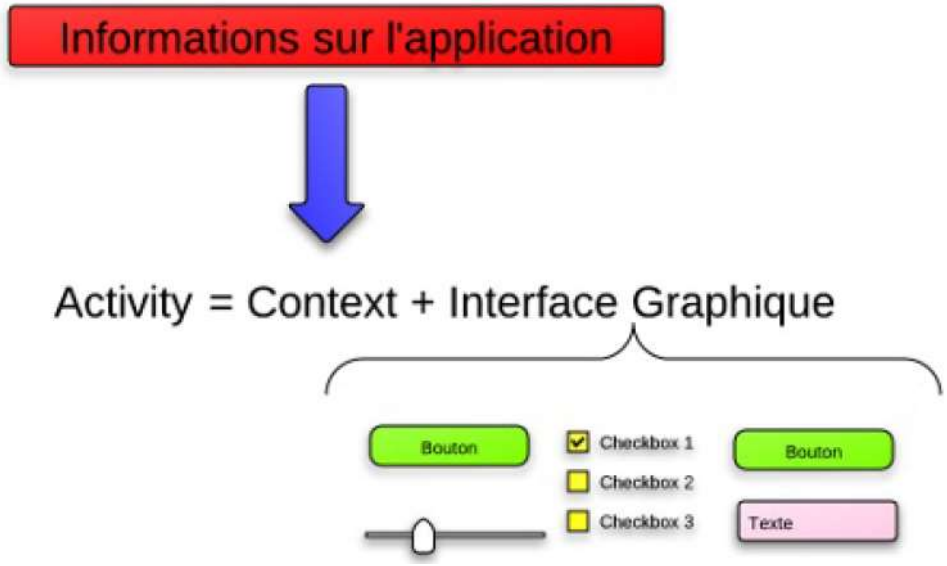
Application interactive

Principe d’application android

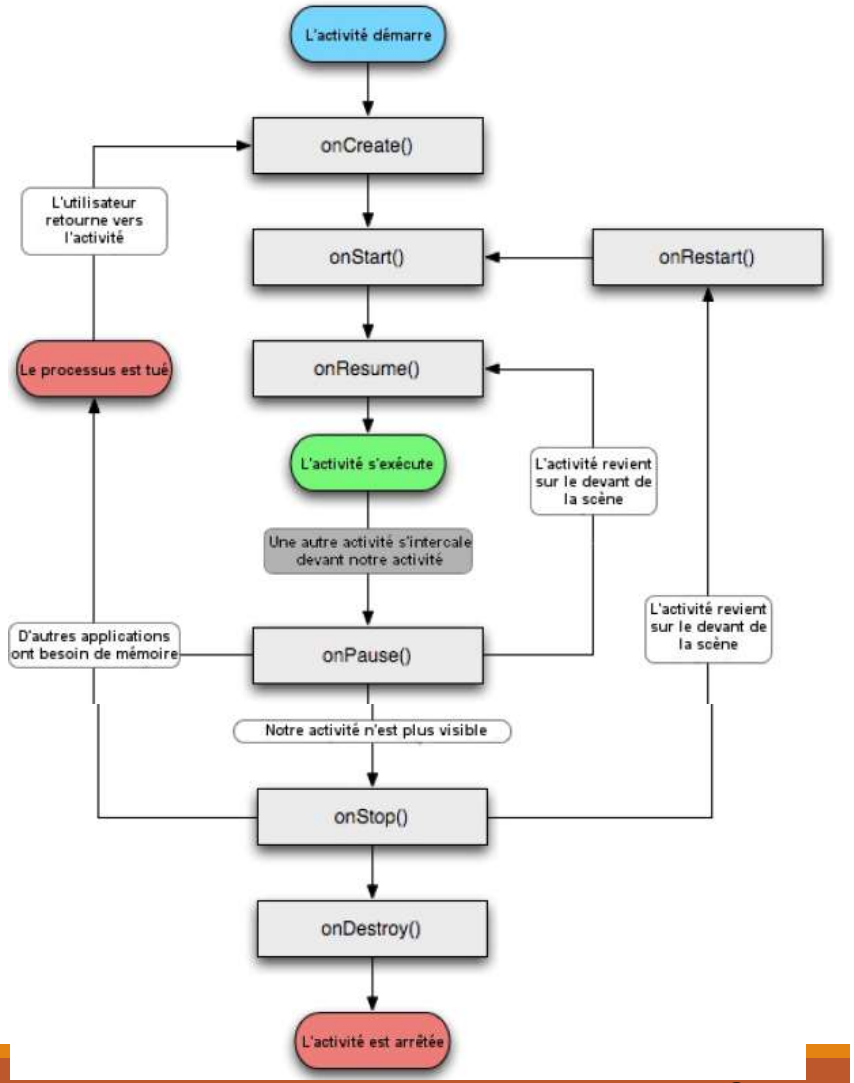
Interface graphique : ensemble d’éléments visuels avec lesquels peuvent interagir les utilisateurs

Contexte : des informations sur l'état actuel de l'application



Cycle de vie d'une activité

- Chaque activité a un ensemble d'états
- rythmé par les interactions avec le système et d'autres applications
- **Le cycle de vie d'une activité : les étapes que va traverser de sa** naissance à sa mort (du «onCreate» à «onDestroy»)



resumed : cad en exécution, visible a l‘écran et interagit avec l'utilisateur

paused : partiellement visible, n'interagit pas, ne s’exécute pas

stopped : en background, n'est plus visible, mais ses données sont figées et maintenues en mémoire.

Accès aux composants graphiques (1)

Attribuer un identifiant aux widgets des layouts

Principe :

```
<TextView
android:id="@+id/text"
android:layout_width="wrap_content"
android:layout_height="wrap_content "
```

- ➔ attribuer l'identifiant « text » au composant TextView
- ➔ on peut accéder à cet élément par son id et agir dessus au travers du code Java

Accès aux composants graphiques (2)

Tout élément défini dans le répertoire **/res** impacte le fichier R.java (à ne pas toucher)

Chemin d'accès aux ressources via R.java

- Interface graphique : R.layout.activity_main
- Image : R.drawable.ic_launcher

Objet java représentant les ressources :

- tt = (TextView)findViewById(R.id.text);

Interface principale

Associer la première vue graphique à l'activité principale de l'application :

```
public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.acceuil);
}
```

« saveInstanceState » : enregistre un ensemble de paramètre d’état de l’application

Manipulation des ressources

Modification d’un texte :

- TextView tt = (TextView)findViewById(R.id.text);
- tt.setText("Le texte de notre TextView");
- String p = tt.getText().toString();
- float f = Float.valueOf(p);

Modification du couleur d’un layout :

- tt.setBackgroundColor(Color.BLACK);

Principe de l’interaction graphique

évènement= une interaction avec l’interface

- onClick()
- onLongClick()
- onFocusChange()
- onTouch()

Listener = un objet qui détecte l'évènement et afin de permettre le traiter

- View.OnClickListener
- View.OnLongClickListener
- View.OnKeyListener

Implémentation par héritage

Implémenter l'interface du listener

Définir le traitement de la méthode

Exemple :

```
public void onCreate(Bundle savedInstanceState) {
…
b = (Button) findViewById(R.id.boutton);
b.setOnClickListener(this);
… }

public void onClick(View v) {
/* Réagir au clic */
}
```

Implémentation par classe anonyme

Avantage : coder uniquement la méthode voulue

Exemple :

```
public void onCreate(Bundle savedInstanceState) {
…
b = (Button) findViewById(R.id.boutton);
b.setOnClickListener( (new OnClickListener( ) {
public void onClick(View v) {
// Réagir à un long clic
}
});
```

Duplication d’évènement

Cas d’un même évènement à appliquer pour plusieurs views

Démarche

Définir la méthode du listener

Utiliser une structure conditionnel en testant l’identifiant du view

Exemple :

```
public void onClick(View v) {
If (v.getId()) == R.id.bouton1 then
 …
Else if (v.getId())== R.id.bouton2 then
…
}
```

Exemple :

Test de RadioButton :

- Donner un identifiant à chaque choix
- Tester lequel des choix est coché

Exemple :

```
<RadioGroup
android:id="@+id/group"
…
android:checkedButton="@+id/radio2"
android:orientation="horizontal "   >
<RadioButton
android:id="@+id/radio1"
…
android:text="Mètre "   />
<RadioButton
android:id="@+id/radio2"
…
android:text="Centimètre"  />   </RadioGroup>
```